

SPARSE-WAM: ACCELERATING WORLD ACTION MODELS VIA ACTION-GUIDED SPARSE IMAGINATION

Xinling Xie^{1,2,*,†} Haodong Wang^{2,*} Mi Jiazhi² Zhiming Liu³ Zicong Hong⁴
 Xiaoyi Pang² Qianli Liu² Yangjia Hu² Ying Chen² Zhengyang Yan² Song Guo²

¹ NJU ² HKUST ³ HIT ⁴ EPFL

* Equal contribution. [†] Work done during an internship at HKUST.

ABSTRACT

World-action models (WAMs) leverage pretrained video models to improve generalization in robot control by jointly predicting future visual states and actions. This capability comes at a substantial inference cost, as dense future-frame tokens are repeatedly processed during denoising. Prior methods address this by token pruning that prioritizes visual fidelity to reduce denoising costs in video diffusion models. However, these methods do not use action relevance to determine which future-frame tokens to retain during joint denoising in WAMs. In this paper, we propose **Sparse-WAM**, a training-free framework for *action-guided sparse imagination* that selectively processes future-frame tokens to accelerate WAM inference. We observe substantial overlap in the spatial distribution of attention from action tokens to future-frame tokens (action-to-future attention) between consecutive denoising steps, despite continued updates to the future representations. Motivated by this, we develop *Action-Guided Token Selection* to retain frame-specific action-relevant regions together with cross-frame context. However, a naive implementation can incur attention-scoring and token-packing overhead that offsets the computational savings from pruning. We therefore introduce **Pilot**, an efficient engine that reduces sparse inference overhead through lightweight scoring and cross-step reuse of token selections. On LIBERO with FastWAM-Joint and RoboLab-120 with Cosmos 3 Edge, **Sparse-WAM** achieves inference speedups of approximately $2.0\times$ and $1.8\times$, respectively, over dense eager inference on an NVIDIA RTX 4090, while largely preserving task performance.

1 INTRODUCTION

World action models (WAMs) have emerged as a promising paradigm for robotic control. Recent WAMs such as DreamZero (Ye et al., 2026b) and Cosmos 3 (NVIDIA, 2026) jointly denoise future visual states and actions, leveraging spatiotemporal priors to improve generalization and robustness (Zhang et al., 2026). However, explicitly generating high-resolution future frames introduces a large number of visual tokens into each denoising step. Since the cost of each denoising step grows significantly with the number of processed tokens (Anagnostidis et al., 2025), repeatedly processing these future-frame tokens increases inference latency and limits control frequency (Guo et al., 2024).

To reduce this repeated computation, existing methods either remove imagination at inference or compress future-frame tokens. The former uses future imagination during training but omits explicit future prediction during action generation (Yuan et al., 2026; Li et al., 2026b; Zhao et al., 2026a). The latter retains future prediction with fewer or lower-fidelity visual tokens (Chen et al., 2026; Li et al., 2026a), but still devotes computation to regions with limited relevance to actions.

Token pruning offers a fine-grained means of reducing the cost of processing dense imagination tokens by selectively retaining a subset for computation. This paradigm has been explored in VLAs to retain useful information from observed inputs (Xu et al., 2025; Wang et al., 2026a) and in video generation to preserve visual quality (Zou et al., 2025; Feng et al., 2026). In WAMs, however, future-frame tokens are initialized from noise and jointly denoised with action tokens, making action-relevant regions difficult to determine in advance.

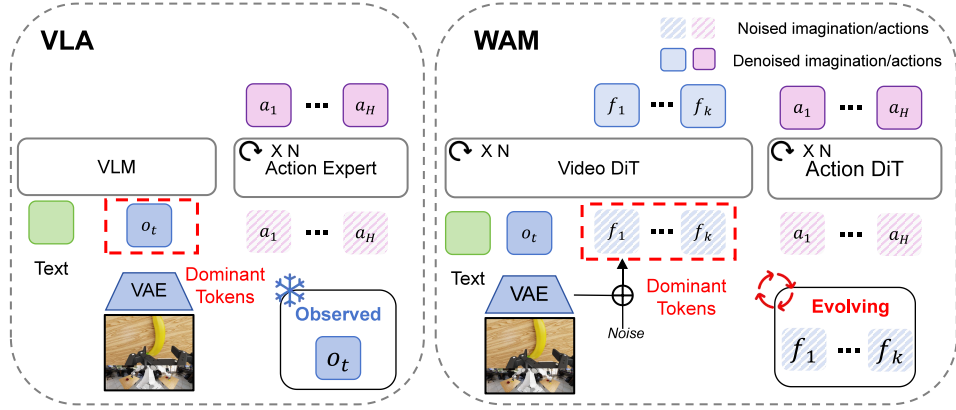


Figure 1: Representative architectures of VLA and WAM. In WAM, noisy imagination tokens constitute the majority of the visual-action sequence and evolve throughout denoising.

To understand how action-relevant regions evolve during denoising, we examine attention from action tokens to future-frame tokens (see subsection 3.2). We observe that, despite continued changes in future representations, the regions attended to by action tokens exhibit substantial spatial overlap between consecutive denoising steps. This consistency offers an opportunity to reuse token selections across steps, but does not imply that the relevant regions remain unchanged throughout denoising. These observations motivate using action-to-future attention to select future regions for sparse computation and reusing the selections across denoising steps. However, attention scoring and token reorganization introduce additional overhead that can offset the computational savings. The core challenge is therefore to identify useful future regions while keeping the cost of selection and sparse execution low.

To address this challenge, we introduce **Sparse-WAM**, a training-free framework for action-guided sparse imagination in WAMs. It uses action-to-future attention to retain frame-specific regions together with shared spatial context. An efficient execution engine combines lightweight scoring with cross-step selection reuse to reduce the cost of joint visual-action inference.

We summarize our contributions as follows:

- We reveal substantial spatial overlap in action-to-future attention between consecutive denoising steps, despite continued changes in future representations. This finding motivates reusing action-guided token selections during joint denoising.
- We propose **Sparse-WAM**, which uses temporally aligned action-to-future attention to select frame-specific regions together with shared spatial context. It concentrates Transformer computation on selected future tokens while retaining all observation and action tokens.
- We develop **Pilot**, an efficient engine for online token selection and sparse execution. It combines lightweight attention profiling with cross-step reuse of selected positions and packing metadata, reducing the overhead of action-guided sparse inference.
- We evaluate Sparse-WAM on three WAMs across LIBERO (Liu et al., 2023), RoboLab-120 (Yang et al., 2026), LIBERO-Plus (Fei et al., 2025), and real-world robotic tasks. On LIBERO, Sparse-WAM achieves a $1.98\times$ speedup over dense eager inference with a 0.30 percentage-point decrease in average task success.

2 RELATED WORK

Efficient World Action Models. Recent work explores action-controllable world modeling (Miao et al., 2026) and combines visual foresight with multimodal VLA reasoning (Shou et al., 2026). Repeated denoising of future-frame tokens makes diffusion-based WAM inference computationally expensive (Ye et al., 2026b; Li et al., 2025; Bi et al., 2025). One line of work retains future prediction during training but generates actions directly from learned world representations at inference (Yuan et al., 2026; Ye et al., 2026a; Li et al., 2026b;c). Another preserves inference-time future modeling

while reducing its cost through compact latent subgoals, lower-resolution representations, asymmetric denoising, or feature reuse (Chen et al., 2026; Li et al., 2026a; Zhao et al., 2026a). Our method retains joint visual–action denoising and selectively allocates future-frame token computation according to action relevance, without additional model training.

Token-Level Caching and Pruning. Token caching and pruning reduce inference cost through feature reuse and token removal. VLA methods use observation similarity and task or action cues (Pei et al., 2025; Liu et al., 2025; Ma et al., 2026), as exemplified by VLA-Cache and SpecPrune-VLA (Xu et al., 2025; Wang et al., 2026a). In video generation and diffusion world models, selective computation aims to preserve generation quality (Liu et al., 2026; Zhang et al., 2025a). ToCa uses token importance for caching (Zou et al., 2025), while WorldCache exploits trajectory predictability for reuse and extrapolation (Feng et al., 2026). However, observed-input relevance and visual fidelity do not directly determine which evolving future regions support action prediction. Our method therefore uses action-to-future attention to guide future-frame token selection and reuses selected positions across denoising steps.

Efficient LLM Serving. Dynamic expert routing and scheduling improve on-device MoE serving efficiency (Wang et al., 2025), while 4-bit quantization reduces LLM memory and computation costs (Wang et al., 2026b; Hu et al., 2026). These approaches optimize expert execution or numerical precision; Sparse-WAM instead selects future-frame tokens during joint visual–action denoising.

3 PRELIMINARIES AND MOTIVATION

3.1 WORLD ACTION MODELS

As shown in Figure 1, WAMs with a shared Transformer sequence jointly denoise future-frame tokens $\mathbf{z}_v$ and an H -step action chunk $\mathbf{a}_{1:H}$ (Ye et al., 2026b; NVIDIA, 2026). Conditioned on the current observation, task instruction, and robot state, successive denoising steps refine the same predicted future and action chunk. During this process, action tokens attend to future-frame tokens, allowing information from the predicted future to inform action updates. Reducing future-frame computation therefore calls for understanding how action tokens access this information.

Action-to-Future Attention. Attention from action queries to future-frame keys provides a token-level view of this interaction. For a predicted future comprising F latent frames with N_s tokens per frame, let $\mathbf{z}_{f,j}$ denote the future-frame token at spatial position j in frame f , and $\mathbf{a}_i$ the i -th action token. At Transformer layer ℓ and denoising step τ (starting from $\tau = 0$), let $A_{\ell,h}^{(\tau)}(i, f, j)$ denote the attention weight from the query of action token $\mathbf{a}_i$ to the key of future-frame token $\mathbf{z}_{f,j}$ in head h . These weights retain the normalization over all keys visible to each action query. We define action-to-future attention by averaging these weights across heads:

$$U_{\ell}^{(\tau)}(i, f, j) = \frac{1}{N_h} \sum_{h=1}^{N_h} A_{\ell,h}^{(\tau)}(i, f, j), \quad (1)$$

where N_h is the number of attention heads, $i = 1, \dots, H$ indexes action tokens, $f = 1, \dots, F$ indexes future frames, and $j = 1, \dots, N_s$ indexes spatial tokens within each frame. A larger value indicates stronger attention from action token i to future-frame token (f, j) . We use this attention as a low-cost proxy for token relevance when allocating sparse computation. We measure cross-frame and cross-step overlap by summing the smaller of the two normalized attention values at each spatial position (see Appendix C.3).

3.2 OBSERVATIONS

This section examines how action-to-future attention is distributed across future frames and spatial positions, and how these patterns vary across Transformer layers and denoising steps. We conduct this analysis on Cosmos3-Edge and Cosmos3-Nano using RoboLab (Yang et al., 2026).

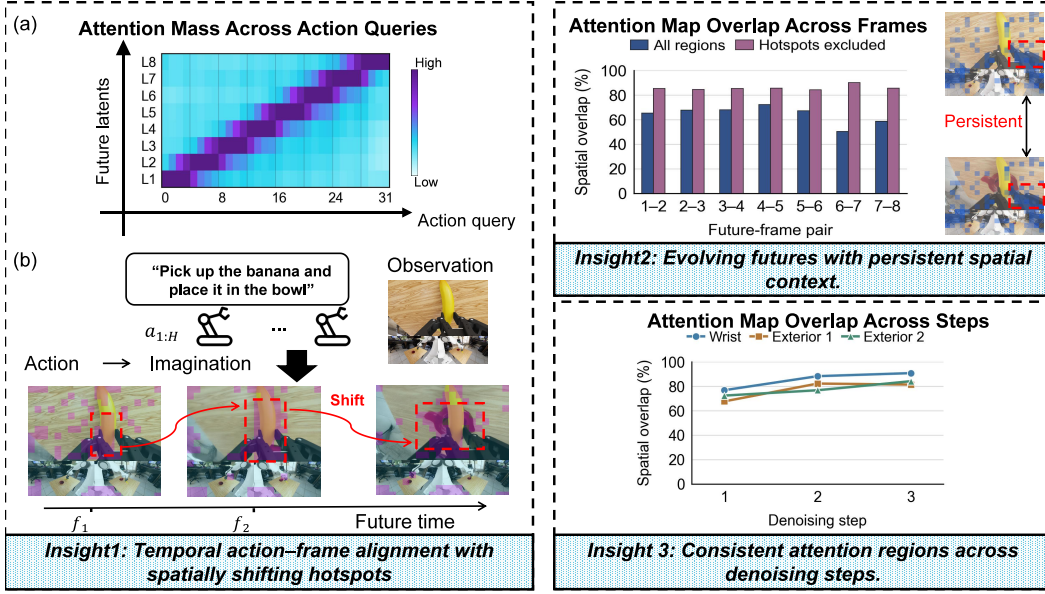


Figure 2: Insight 1: (a) Action queries exhibit temporal alignment with future frames, with attention extending to neighboring frames near temporal group boundaries. (b) Action-relevant hotspots shift spatially across future frames. Insight 2: Excluding hotspots yields higher attention overlap between adjacent frames. Insight 3: Attention maps exhibit substantial spatial overlap between consecutive denoising steps in all three camera views.

Action-Frame Alignment and Spatial Variation. We identify two complementary patterns in action-to-future attention. First, Figure 2 (Insight 1(a)) shows an approximately diagonal band in the frame-level attention map: each future frame predominantly receives attention from a contiguous group of roughly H/F action queries. Queries near group boundaries also attend to neighboring frames, indicating that the alignment extends across temporal group boundaries. Second, Figure 2 (Insight 1(b)) shows that attention hotspots shift spatially across future frames. These patterns motivate scoring each future frame using its temporally associated action queries and selecting spatial regions separately across frames.

Cross-Frame Context Consistency. We further compare cross-frame attention overlap with and without hotspot regions. As shown in Figure 2 (Insight 2), overlap increases after hotspot exclusion, indicating greater consistency in the spatial distribution of the remaining attention. The visualizations also show attention to contextual regions, including parts of the gripper outside the object-contact area. Together, these observations suggest that persistent spatial context complements frame-specific hotspots, motivating shared anchor positions alongside per-frame token selection.

Cross-Step Attention Consistency. We next examine how spatial attention distributions change across denoising steps that refine the same predicted future. As shown in Figure 2 (Insight 3), the distributions exhibit substantial overlap between consecutive steps in each of the three camera views. Thus, future representations continue to evolve while their spatial attention distributions remain relatively consistent. This observation motivates reusing token selections across denoising steps to reduce repeated selection overhead. Appendix C.3 defines the overlap measure and reports quantitative cross-step and cross-frame comparisons.

4 METHODOLOGY

As illustrated in Figure 3, Sparse-WAM uses action-to-future attention to allocate computation within imagined futures. In subsection 4.1, we combine frame-specific core tokens with shared spatial anchors to retain changing attention hotspots and persistent context. In subsection 4.2, we introduce

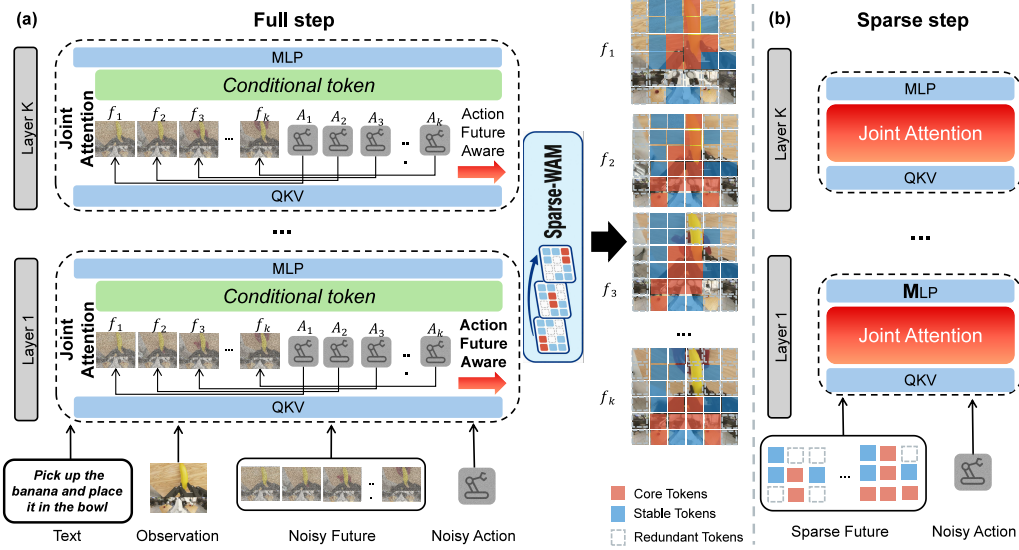


Figure 3: Overview of Sparse-WAM. Future-frame token positions are selected using action-to-future attention and reused during sparse denoising.

Pilot, an engine that combines lightweight attention profiling with cross-step reuse to execute these selections efficiently.

4.1 ONLINE ACTION-GUIDED TOKEN SELECTION

As discussed in Figure 2, the token selection needs to follow the hotspots of each future frame while retaining contextual regions shared across frames. We address these complementary needs with K_c frame-specific core tokens and K_s shared spatial anchors per frame, where $K_c + K_s \leq N_s$. Selection uses attention from the dense conditional forward at a full denoising step τ_d . For brevity, we write $U_\ell(i, f, j) = U_\ell^{(\tau_d)}(i, f, j)$ below.

Scoring Action-Relevant Regions. The temporal alignment in Figure 2 (Insight 1(a)) suggests that each future frame should be scored using its associated action queries. Assuming that H is divisible by F , we partition the H queries into F consecutive groups of size H/F , denoting the group for frame f by $\mathcal{I}_f$. The spatial score at layer ℓ is

$$S_\ell(f, j) = \sum_{i \in \mathcal{I}_f} \alpha_{f,i} U_\ell(i, f, j), \quad (2)$$

where the nonnegative weights $\alpha_{f,i}$ sum to one. Central queries receive larger weights because queries near group boundaries also attend to neighboring frames.

To identify layers whose attention provides localized signals for token selection, we assign each layer a score $Q_\ell = R_\ell(1 - E_\ell)$. Here, R_ℓ measures frame-aligned attention mass, and E_ℓ is the normalized spatial entropy averaged across frames (Zhang et al., 2025b). We select the K_{layer} highest-scoring layers, forming $\mathcal{L}_{\text{core}}$, and aggregate their spatial scores:

$$V(f, j) = \frac{\sum_{\ell \in \mathcal{L}_{\text{core}}} Q_\ell S_\ell(f, j)}{\sum_{\ell \in \mathcal{L}_{\text{core}}} Q_\ell}. \quad (3)$$

Appendix A.1 provides the query-group definition and layer-quality calculations.

Frame-Specific Core Tokens. Since attention hotspots shift across future frames, each frame selects its own core positions using $V(f, j)$. We first identify positions with high core scores in at least one future frame, then select frame-specific tokens from this common candidate pool.

Let $\mathcal{J} = \{1, \dots, N_s\}$ denote all spatial positions. The candidate pool contains $N_s - K_s$ positions, preserving capacity for the shared anchors. We construct the pool and select core positions as

$$\mathcal{P} = \text{TopK}_{j \in \mathcal{J}} \left(\max_f V(f, j), N_s - K_s \right), \quad (4)$$

$$\mathcal{C}_f = \text{TopK}_{j \in \mathcal{P}} (V(f, j), K_c), \quad (5)$$

where TopK returns the indices of the largest scores. The maximum across frames determines the common candidate pool, while each frame’s own scores determine its core selection, allowing the retained positions to follow spatially shifting hotspots.

Shared Spatial Anchor Tokens. The increased cross-frame attention overlap after hotspot exclusion (Figure 2, Insight 2) suggests that contextual regions receive more consistent attention across future frames. We therefore complement frame-specific core tokens with shared spatial anchors.

To avoid duplicating core positions, anchors are selected from $\mathcal{E}$, the positions not selected as core tokens in any frame. Because all core selections lie within the common pool $\mathcal{P}$ of size $N_s - K_s$, at least K_s positions remain available for anchors. Appendix A.1 provides the formal candidate-set definition and availability guarantee.

To rank these candidates, we aggregate $S_\ell(f, j)$ over all layers using the quality weights Q_ℓ , including signals beyond the layers selected for core localization. Let μ_j and CV_j denote the cross-frame mean and coefficient of variation of this aggregated score at position j . We select positions with strong and consistent attention:

$$\mathcal{S} = \text{TopK}_{j \in \mathcal{E}} \left(\frac{\mu_j}{1 + \text{CV}_j}, K_s \right). \quad (6)$$

The numerator rewards attention strength, while the denominator penalizes cross-frame variation. Each frame retains the positions $\mathcal{C}_f \cup \mathcal{S}$. The two sets are disjoint, giving exactly $K_c + K_s$ tokens per frame and a fixed sequence length for sparse execution. Only anchor positions are shared: their representations remain frame-specific and are updated separately.

4.2 PILOT: ONLINE SELECTION AND SPARSE EXECUTION

In this section, we implement sparse execution through lightweight attention profiling, cross-step selection reuse, and cached prediction updates. Together, these mechanisms reduce selection and execution overhead while maintaining the sampling updates required by joint visual–action denoising.

Low-Overhead Attention Profiling. At each full step, Pilot obtains the required scores from the dense conditional forward without an additional network forward. The scorer computes only the query–key products between each future frame and its associated action–query group. It reuses the original log-sum-exp normalizers computed over all visible keys, preserving the attention mass used in layer-quality scoring. The resulting probabilities are aggregated directly into $S_\ell(f, j)$, avoiding materialization of the full attention matrix or an additional value-weighted attention output. Appendix A.2 provides the extraction formula and implementation details.

Cross-Step Reuse and Compact Execution. The observed cross-step attention consistency motivates reusing selections within each action chunk. A full step constructs the selection and caches the visual velocity predictions; subsequent sparse steps reuse both. Given the short denoising schedules of the evaluated WAMs, our default configuration uses only the initial step for full computation. Selections and cached predictions are recomputed for each new action chunk. Alternative refresh schedules are evaluated in Appendix C.2.

During sparse steps, the selected future tokens are processed as a compact sequence together with all observation and action tokens. Pilot reuses their original-position indices and compatible packing metadata across Transformer layers and denoising steps. The fixed per-frame token budget maintains a constant sequence length despite different spatial selections across frames.

Sampling Updates for Omitted Regions. Future tokens omitted from Transformer computation still participate in the sampling process. Let τ_d denote the most recent full step and $\mathbf{M}_v$ the retained-position mask mapped to the full future-latent grid. Pilot combines current predictions at retained

Table 1: Performance and inference efficiency on LIBERO.

Method	Success Rate (%)				Avg. SR (%)	Avg. Speedup	FLOPs
	Spatial	Object	Goal	Long			
FastWAM-Joint	99.00%	100.00%	98.20%	97.80%	98.75%	1.00×	100.00%
+ ToCa	98.80%	99.80%	98.60%	98.60%	98.95%	1.09×	69.19%
+ C ³ ache	98.60%	99.40%	97.60%	97.40%	98.25%	1.30×	75.00%
+ SpecPrune-VLA	97.20%	99.80%	96.80%	92.40%	96.55%	1.43×	73.53%
+ WorldCache	99.60%	99.80%	98.00%	98.60%	99.00%	1.62×	60.00%
+ Sparse-WAM	98.80%	99.20%	98.60%	97.20%	98.45%	1.98×	49.65%

positions with cached predictions elsewhere:

$$\tilde{\mathbf{v}}_v^{(\tau)} = \mathbf{M}_v \odot \mathbf{v}_v^{(\tau)} + (\mathbf{1} - \mathbf{M}_v) \odot \mathbf{v}_v^{(\tau_d)}, \quad \tau > \tau_d, \quad (7)$$

where $\odot$ denotes element-wise multiplication, $\mathbf{v}_v^{(\tau)}$ contains current predictions scattered to their retained positions in the full grid, and $\mathbf{v}_v^{(\tau_d)}$ is the cached prediction from the full step. The original sampler uses $\tilde{\mathbf{v}}_v^{(\tau)}$ to update all future latents, including omitted regions, while action predictions are recomputed at every step.

5 EXPERIMENTS

5.1 EXPERIMENTAL SETTINGS

Benchmarks and Models. We evaluate Sparse-WAM using FastWAM-Joint (Yuan et al., 2026) on LIBERO (Liu et al., 2023) and LIBERO-Plus (Fei et al., 2025), and Cosmos 3 Nano Policy and Cosmos 3 Edge (NVIDIA, 2026) on RoboLab-120 (Yang et al., 2026). LIBERO comprises four manipulation suites, LIBERO-Plus adds seven perturbation categories, and RoboLab-120 contains 120 tasks across three difficulty levels. We also evaluate FastWAM-Joint on real-world object packing, cup stacking, and battery insertion tasks. Figure 4 illustrates representative tasks. Appendix B.1 provides benchmark protocols.

Baselines and Evaluation Setup. We compare against four inference acceleration methods: ToCa (Zou et al., 2025), WorldCache (Feng et al., 2026), SpecPrune-VLA (Wang et al., 2026a), and C³ache (Zhao et al., 2026b). All policy inference runs on an NVIDIA RTX 4090, with latency measured using CUDA events. Appendix B.2 provides implementation details, while Appendix B.3 describes the baseline adaptations.

Evaluation Metrics. We report task success rate (SR), inference speedup, and FLOPs as a percentage of the corresponding dense model. Speedups in the main tables compare optimized accelerated inference against dense eager inference and include gains from both the acceleration methods and execution optimizations. Appendix B.4 details the timing protocol and reports on an execution backend ablation and comparisons with matched execution backends.

5.2 SIMULATION RESULTS

Results on LIBERO. Table 1 reports performance and inference efficiency across the four LIBERO suites. Speedups in both main tables are measured relative to dense eager inference and reflect the combined effects of each acceleration method and execution optimizations. Sparse-WAM achieves the largest speedup among the evaluated methods (1.98×), reducing FLOPs by 50.35% while attaining 98.45% success, 0.30 percentage points below dense inference. ToCa, WorldCache, and C³ache also maintain high success rates but achieve smaller speedups, up to 1.62×. SpecPrune-VLA incurs a larger performance drop, particularly on long-horizon tasks.

Results on RoboLab-120. Table 2 shows that Sparse-WAM achieves the highest average success rates among the accelerated variants on both Cosmos backbones: 23.00% on Edge and 35.50%

Table 2: Performance and inference efficiency on RoboLab-120.

Method	Success Rate (%)			Avg. SR (%)	Avg. Speedup	FLOPs
	Simple	Moderate	Complex			
Cosmos3-Edge	25.60%	23.30%	11.80%	22.90%	1.00×	100.00%
+ ToCa	9.84%	14.10%	1.18%	10.00%	1.26×	65.68%
+ C ³ ache	9.06%	12.82%	0.59%	9.08%	1.51×	75.14%
+ SpecPrune-VLA	10.31%	14.10%	0.00%	10.08%	1.52×	50.35%
+ WorldCache	21.56%	21.03%	11.18%	19.92%	1.52×	75.14%
+ Sparse-WAM	25.00%	25.90%	8.82%	23.00%	1.85×	60.73%
Cosmos3-Nano-Policy	40.63%	35.38%	25.29%	36.75%	1.00×	100.00%
+ ToCa	9.69%	12.82%	0.00%	9.33%	1.33×	65.21%
+ C ³ ache	16.09%	13.08%	1.18%	13.00%	1.48×	75.00%
+ SpecPrune-VLA	27.50%	23.08%	11.76%	23.83%	1.88×	54.79%
+ WorldCache	36.56%	30.00%	20.00%	32.08%	1.48×	75.00%
+ Sparse-WAM	40.16%	32.82%	24.12%	35.50%	1.81×	59.78%

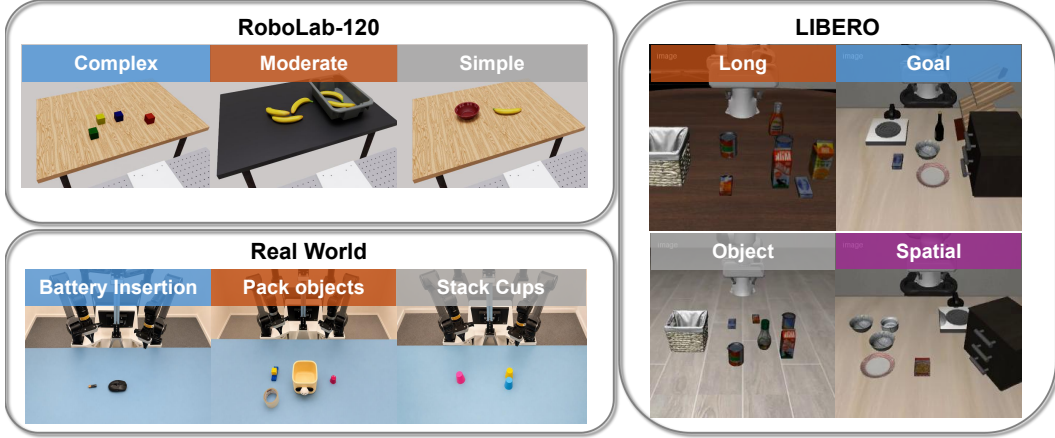


Figure 4: Tasks on LIBERO, RoboLab-120 and Real World.

on Nano, compared with 22.90% and 36.75% for dense inference. With execution optimizations enabled, it achieves respective speedups of $1.85\times$ and $1.81\times$ over dense eager inference. WorldCache preserves performance better than ToCa and C³ache but is slower than Sparse-WAM. SpecPrune-VLA is slightly faster on Nano ($1.88\times$), at a success rate 11.67 percentage points below ours. On Edge, Sparse-WAM outpaces our SpecPrune-VLA adaptation ($1.85\times$ versus $1.52\times$) despite using more FLOPs (60.73% versus 50.35% of dense inference), showing that FLOPs alone do not determine latency. Our implementation reuses token selections and maintains fixed sparse sequence lengths to support compiled execution. Appendix B.4 provides backend ablations and matched-backend comparisons on Cosmos 3 Edge; Appendix C.1 presents LIBERO-Plus robustness results across seven perturbation categories.

5.3 REAL-WORLD EXPERIMENTS

We evaluate Sparse-WAM on the AgileX Cobot Magic platform, which is equipped with three cameras providing different viewpoints: one primary camera and two wrist-mounted cameras. We fine-tune FastWAM-Joint using our collected data, following the configuration detailed in Appendix B.5.

We design three tasks: object packing, cup stacking, and battery insertion, as shown in Figure 4. Table 3 reports the real-world performance of Sparse-WAM. Our method achieves a $2.08\times$ speedup with an average success rate of 75.00%, compared with 77.78% for dense FastWAM-Joint. These

Table 3: Real-world manipulation performance on AgileX Cobot Magic.

Method	Success Rate (%)				Latency (ms)	Speedup
	Pack objects	Stack cups	Battery assembly	Average		
FastWAM-Joint	75.00	75.00	83.33	77.78	501	1.00×
+ Sparse-WAM	83.33	75.00	66.67	75.00	242	2.08×

results demonstrate the potential of Sparse-WAM to accelerate real-world WAM inference by reducing redundant computation over imagination tokens while largely preserving average task success.

5.4 ABLATION STUDIES

Component Ablation. Figure 5a compares token selection strategies on Cosmos 3 Nano Policy, with all sparse variants retaining 184 of 360 tokens per future frame. Combining $K_c = 80$ core tokens with $K_s = 104$ stable anchors achieves a 35.5% success rate, outperforming both core-only and stable-only selection. Random selection achieves only 31.7%, compared with 36.8% under dense inference. The combined variant maintains approximately $1.81\times$ speedup, comparable to core-only selection. These results support the complementary roles of frame-specific core tokens and shared stable anchors in preserving action-relevant imagination, with little additional inference overhead.

Pruning Ratio. Figure 5b examines the effect of future-token pruning on Cosmos 3 Nano Policy. We define the future-token pruning ratio as $r = 1 - K_{\text{ret}}/N_s$, where K_{ret} is the number of retained future tokens per frame. Increasing the pruning ratio from 28.89% to 68.89% improves speedup from $1.45\times$ to $2.36\times$, while success rate decreases from 37.5% to 28.3%. These results illustrate the efficiency–control trade-off: more aggressive pruning accelerates inference at the expense of task success.

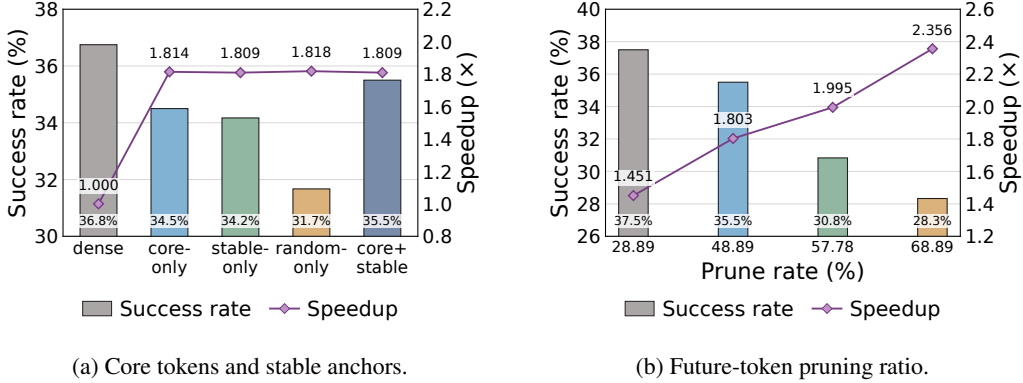


Figure 5: Ablation studies of token selection components and future-token pruning ratios.

6 LIMITATIONS AND FUTURE WORK

Future work could broaden the evaluation and deployment of Sparse-WAM in three directions. First, our inference benchmarks were conducted on an NVIDIA RTX 4090; extending evaluation to other GPUs and resource-constrained devices would help assess its efficiency across hardware platforms. Second, we evaluated three WAMs, and further studies could examine its applicability to a broader range of WAM architectures and backbones. Third, the performance degradation observed under environmental perturbations motivates adaptive token-selection mechanisms that adjust to changing conditions to improve robustness while maintaining inference efficiency.

7 CONCLUSION

In this paper, we presented Sparse-WAM, a training-free method that prunes evolving imagination tokens online during joint visual-action denoising. Evaluations on Cosmos 3 Edge, Cosmos 3 Nano Policy, and FastWAM-Joint across LIBERO, RoboLab-120, and real-world manipulation tasks demonstrate its inference efficiency while largely preserving task success. Sparse-WAM achieves approximately $1.8\times$ speedup on RoboLab-120 and $2.0\times$ on LIBERO. These results indicate that dense computation over evolving imagination tokens in WAMs is partially redundant for action prediction. Sparsifying this computation enables more efficient inference while largely preserving task performance.

REFERENCES

- Sotiris Anagnostidis, Gregor Bachmann, Yeongmin Kim, Jonas Kohler, Markos Georgopoulos, Artsiom Sanakoyeu, Yuming Du, Albert Pumarola, Ali K. Thabet, and Edgar Schönfeld. FlexiDiT: Your diffusion transformer can easily generate high-quality samples with less compute. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 28316–28326, 2025. doi: 10.1109/CVPR52734.2025.02637.
- Hongzhe Bi, Hengkai Tan, Shenghao Xie, Zeyuan Wang, Shuhe Huang, Haitian Liu, Ruowen Zhao, Yao Feng, Chendong Xiang, Yinze Rong, Hongyan Zhao, Hanyu Liu, Zhizhong Su, Lei Ma, Hang Su, and Jun Zhu. Motus: A unified latent action world model. *arXiv preprint arXiv:2512.13030*, 2025. URL <https://arxiv.org/abs/2512.13030>.
- Jialei Chen, Kai Wang, Kang Chen, Shuaihang Chen, Feng Gao, Wenhao Tang, Zhiyuan Li, Weilin Liu, Zhuyu Yao, Boxun Li, Yuanbo Xu, and Chao Yu. LaWAM: Latent world action models for efficient dynamics-aware robot policies. *arXiv preprint arXiv:2606.15768*, 2026. URL <https://arxiv.org/abs/2606.15768>.
- Senyu Fei, Siyin Wang, Junhao Shi, Zihao Dai, Jikun Cai, Pengfang Qian, Li Ji, Xinzhe He, Shiduo Zhang, Zhao Ye Fei, Jinlan Fu, Jingjing Gong, and Xipeng Qiu. LIBERO-Plus: In-depth robustness analysis of vision-language-action models. *arXiv preprint arXiv:2510.13626*, 2025. URL <https://arxiv.org/abs/2510.13626>.
- Weilun Feng, Guoxin Fan, Haotong Qin, Mingqiang Wu, Yuqi Li, Xiangqi Li, Zhulin An, Libo Huang, Dingrui Wang, Longlong Liao, Michele Magno, Yongjun Xu, and Chuanguang Yang. WorldCache: Accelerating world models for free via heterogeneous token caching. *arXiv preprint arXiv:2603.06331*, 2026. URL <https://arxiv.org/abs/2603.06331>.
- Yanjiang Guo, Yucheng Hu, Jianke Zhang, Yen-Jen Wang, Xiaoyu Chen, Chaochao Lu, and Jianyu Chen. Prediction with action: Visual policy learning via joint denoising process. In *Advances in Neural Information Processing Systems*, volume 37, pp. 112386–112410. Curran Associates, Inc., 2024. URL https://papers.nips.cc/paper_files/paper/2024/hash/cbe25fa0e7c7084049276888a09acc8d-Abstract-Conference.html.
- Yangjia Hu, Haodong Wang, Zicong Hong, Qianli Liu, Quanxin Shou, Jian Lin, Song Guo, Xiaowei Shen, Xiangjun Huang, Dian Wang, and Jian Yang. MosaicQuant: Inlier-outlier disaggregation for unified 4-bit LLM quantization. *arXiv preprint arXiv:2606.15652*, 2026. URL <https://arxiv.org/abs/2606.15652>.
- Jiajun Li, Tiecheng Guo, Yifan Ye, Rongyu Zhang, Xiaowei Chi, Qianpu Sun, Ying Li, Yunfan Lou, Yan Huang, Zhihe Lu, Meng Guo, and Shanghang Zhang. Efficient-WAM: A 1B-parameter world-action model with low-cost future imagination. *arXiv preprint arXiv:2606.10040*, 2026a. URL <https://arxiv.org/abs/2606.10040>.
- Jingyu Li, Zhe Liu, Dongnan Hu, Junjie Wu, Zipei Ma, Wenxiao Wu, Chao Han, Zhihui Hao, Zhikang Liu, Kun Zhan, Jiankang Deng, Xiatian Zhu, and Li Zhang. Metis: A generalizable and efficient world-action model for autonomous driving and urban navigation. *arXiv preprint arXiv:2606.15869*, 2026b. doi: 10.48550/arXiv.2606.15869. URL <https://arxiv.org/abs/2606.15869>.

-
- Shuang Li, Yihuai Gao, Dorsa Sadigh, and Shuran Song. Unified video action model. In *Proceedings of Robotics: Science and Systems*, 2025. doi: 10.15607/RSS.2025.XXI.074. URL <https://www.roboticsproceedings.org/rss21/p074.html>.
- Ziang Li, Dongzhou Cheng, Yibin Wang, Shiyue Wang, Xiaoyang Xu, Lingxuan Weng, Juan Wang, and Jiaqi Wang. Light-WAM: Efficient world action models with state-fusion action decoding. *arXiv preprint arXiv:2606.08242*, 2026c. URL <https://arxiv.org/abs/2606.08242>.
- Bo Liu, Yifeng Zhu, Chongkai Gao, Yihao Feng, Qiang Liu, Yuke Zhu, and Peter Stone. LIBERO: Benchmarking knowledge transfer for lifelong robot learning. In *Advances in Neural Information Processing Systems*, volume 36, pp. 44776–44791, 2023.
- Ziming Liu, Yifan Yang, Chengruidong Zhang, Yiqi Zhang, Lili Qiu, Yang You, and Yuqing Yang. Region-adaptive sampling for diffusion transformers. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 2346–2356, 2026.
- Ziyan Liu, Yeqiu Chen, Hongyi Cai, Tao Lin, Shuo Yang, Zheng Liu, and Bo Zhao. VLA-Pruner: Temporal-aware dual-level visual token pruning for efficient vision-language-action inference. *arXiv preprint arXiv:2511.16449*, 2025. URL <https://arxiv.org/abs/2511.16449v1>.
- Shilin Ma, Chubin Zhang, Changyuan Wang, Yuji Wang, Yue Wu, Zixuan Wang, Jingqi Tian, Zheng Zhu, and Yansong Tang. SAFE-Pruner: Semantic attention-guided future-aware token pruning for efficient vision-language-action manipulation. *arXiv preprint arXiv:2605.29662*, 2026. URL <https://arxiv.org/abs/2605.29662>.
- Yikun Miao, Fangqi Zhu, Quanxin Shou, Xiaoyi Pang, Zhengyang Yan, Junhao Li, Haodong Wang, Zicong Hong, and Song Guo. OnlineWM: Causality-aware active online learning for effective world modeling. *arXiv preprint arXiv:2609.23753*, 2026. URL <https://arxiv.org/abs/2609.23753>.
- NVIDIA. Cosmos 3: Omnimodal world models for physical AI. *arXiv preprint arXiv:2606.02800*, 2026. URL <https://arxiv.org/abs/2606.02800>.
- Xiaohuan Pei, Yuxing Chen, Siyu Xu, Yunke Wang, Yuheng Shi, and Chang Xu. Action-aware dynamic pruning for efficient vision-language-action manipulation. *arXiv preprint arXiv:2509.22093*, 2025. URL <https://arxiv.org/abs/2509.22093>.
- Quanxin Shou, Fangqi Zhu, Shawn Chen, Puxin Yan, Zhengyang Yan, Yikun Miao, Xiaoyi Pang, Zicong Hong, Ruikai Shi, Hao Huang, Jie Zhang, and Song Guo. HALO: A unified vision-language-action model for embodied multimodal chain-of-thought reasoning. *arXiv preprint arXiv:2602.21157*, 2026. URL <https://arxiv.org/abs/2602.21157>.
- Hanzhen Wang, Jiaming Xu, Yushun Xiang, Jiayi Pan, Yongkang Zhou, Yong-Lu Li, and Guohao Dai. SpecPrune-VLA: Accelerating vision-language-action models via action-aware self-speculative pruning. In *Proceedings of the International Conference on Machine Learning*, 2026a. URL <https://arxiv.org/abs/2509.05614>.
- Haodong Wang, Qihua Zhou, Zicong Hong, and Song Guo. D2MoE: Dual routing and dynamic scheduling for efficient on-device MoE-based LLM serving. In *Proceedings of the 31st Annual International Conference on Mobile Computing and Networking*, ACM MOBICOM '25, pp. 574–588. Association for Computing Machinery, 2025. doi: 10.1145/3680207.3723493. URL <https://doi.org/10.1145/3680207.3723493>.
- Haodong Wang, Junjie Liu, Zicong Hong, Qianli Liu, Jian Lin, Song Guo, and Xu Chen. TwinQuant: Learnable subspace decomposition for 4-bit LLM quantization. *arXiv preprint arXiv:2606.01556*, 2026b. URL <https://arxiv.org/abs/2606.01556>.
- Siyu Xu, Yunke Wang, Chenghao Xia, Dihao Zhu, Tao Huang, and Chang Xu. VLA-Cache: Efficient vision-language-action manipulation via adaptive token caching. *arXiv preprint arXiv:2502.02175*, 2025. URL <https://arxiv.org/abs/2502.02175>.

-
- Xuning Yang, Rishit Dagli, Alex Zook, Hugo Hadfield, Ankit Goyal, Stan Birchfield, Fabio Ramos, and Jonathan Tremblay. RoboLab: A high-fidelity simulation benchmark for analysis of task generalist policies. *arXiv preprint arXiv:2604.09860*, 2026. URL <https://arxiv.org/abs/2604.09860>.
- Angen Ye, Boyuan Wang, Chaojun Ni, Guan Huang, Guosheng Zhao, Hao Li, Hengtao Li, Jie Li, Jindi Lv, Jingyu Liu, Min Cao, Peng Li, Qiuping Deng, Wenjun Mei, Xiaofeng Wang, Xinze Chen, Xinyu Zhou, Yang Wang, Yifan Chang, Yifan Li, Yukun Zhou, Yun Ye, Zhichao Liu, and Zheng Zhu. GigaWorld-Policy: An efficient action-centered world-action model. *arXiv preprint arXiv:2603.17240*, 2026a. URL <https://arxiv.org/abs/2603.17240>.
- Seonghyeon Ye, Yunhao Ge, Kaiyuan Zheng, Shenyuan Gao, Sihyun Yu, George Kurian, Suneel Indupuru, You Liang Tan, Chuning Zhu, Jiannan Xiang, Ayaan Malik, Kyungmin Lee, William Liang, Nadun Ranawaka, Jiasheng Gu, Yinzhen Xu, Guanzhi Wang, Fengyuan Hu, Avnish Narayan, Johan Bjorck, Jing Wang, Gwanghyun Kim, Dantong Niu, Ruijie Zheng, Yuqi Xie, Jimmy Wu, Qi Wang, Ryan Julian, Danfei Xu, Yilun Du, Yevgen Chebotar, Scott Reed, Jan Kautz, Yuke Zhu, Linxi "Jim" Fan, and Joel Jang. World action models are zero-shot policies. *arXiv preprint arXiv:2602.15922*, 2026b. URL <https://arxiv.org/abs/2602.15922>.
- Tianyuan Yuan, Zibin Dong, Yicheng Liu, and Hang Zhao. Fast-WAM: Do world action models need test-time future imagination? *arXiv preprint arXiv:2603.16666*, 2026. URL <https://arxiv.org/abs/2603.16666>.
- Evelyn Zhang, Jiayi Tang, Xuefei Ning, and Linfeng Zhang. Training-free and hardware-friendly acceleration for diffusion models via similarity-based token pruning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 39, pp. 9878–9886, 2025a. doi: 10.1609/aaai.v39i9.33071.
- Zhanguang Zhang, Zhiyuan Li, Behnam Rahmati, Rui Heng Yang, Yintao Ma, Amir Rasouli, Sajjad Pakdamansavaji, Yangzheng Wu, Lingfeng Zhang, Tongtong Cao, Feng Wen, Xinyu Wang, Xingyue Quan, and Yingxue Zhang. Do world action models generalize better than VLAs? A robustness study. *arXiv preprint arXiv:2603.22078*, 2026. URL <https://arxiv.org/abs/2603.22078>.
- Zhisong Zhang, Yan Wang, Xinting Huang, Tianqing Fang, Hongming Zhang, Chenlong Deng, Shuaiyi Li, and Dong Yu. Attention entropy is a key factor: An analysis of parallel context encoding with full-attention-based pre-trained language models. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 9840–9855, 2025b. doi: 10.18653/v1/2025.acl-long.485. URL <https://aclanthology.org/2025.acl-long.485/>.
- Weiheng Zhao, Haoyi Jiang, Xin Shi, Liu Liu, Fan Huang, Zhizhong Su, Wei Sui, and Xinggang Wang. Faster-WAM: Efficient inference-time future conditioning for robust world action models. *arXiv preprint arXiv:2608.04404*, 2026a. URL <https://arxiv.org/abs/2608.04404>.
- Weisen Zhao, Lam Nguyen, Zhicong Lu, and Yuzhang Shang. C³ache: Accelerating world action models with cross inference chunk cache. *arXiv preprint arXiv:2606.08962*, 2026b. URL <https://arxiv.org/abs/2606.08962>.
- Chang Zou, Xuyang Liu, Ting Liu, Siteng Huang, and Linfeng Zhang. Accelerating diffusion transformers with token-wise feature caching. In *International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=yYZbZGo4ei>.

A ADDITIONAL METHOD DETAILS

We give the definitions needed to implement the selection rule in subsection 4.1 and the scorer in subsection 4.2. All scores below are evaluated at a full denoising step τ_d ; we suppress this index.

A.1 TOKEN SELECTION DETAILS

Queries and layer quality. For H action queries and F future frames, with H divisible by F , the disjoint frame-aligned query groups are

$$\mathcal{I}_f = \{(f-1)H/F + 1, \dots, fH/F\}, \quad f = 1, \dots, F. \quad (8)$$

The weights in Equation 2 are nonnegative and sum to one within each group. Central queries receive greater weight; the exact settings are given in Appendix B.2. For the resulting spatial scores $S_\ell(f, j)$, define

$$\begin{aligned} R_\ell &= \frac{1}{F} \sum_{f,j} S_\ell(f, j), & \bar{S}_\ell(f, j) &= \frac{S_\ell(f, j)}{\sum_{j'} S_\ell(f, j')}, \\ E_\ell &= -\frac{\sum_{f,j} \bar{S}_\ell(f, j) \log \bar{S}_\ell(f, j)}{F \log N_s}, & Q_\ell &= R_\ell(1 - E_\ell). \end{aligned} \quad (9)$$

Here R_ℓ is frame-aligned attention mass and E_ℓ is normalized spatial entropy (Zhang et al., 2025b), with $0 \log 0 = 0$. The K_{layer} highest-quality layers form $\mathcal{L}_{\text{core}}$ in Equation 3.

Shared anchors. With the common pool $\mathcal{P}$ and per-frame core sets $\mathcal{C}_f$ from Equations 4–5, the unused spatial positions satisfy

$$\mathcal{E} = \mathcal{J} \setminus \bigcup_f \mathcal{C}_f, \quad |\mathcal{E}| \geq N_s - |\mathcal{P}| = K_s. \quad (10)$$

This guarantees enough candidates for the shared anchors. Their scores use all layers, rather than only $\mathcal{L}_{\text{core}}$:

$$\begin{aligned} V_{\text{all}}(f, j) &= \frac{\sum_\ell Q_\ell S_\ell(f, j)}{\sum_\ell Q_\ell}, \\ \mu_j &= \frac{1}{F} \sum_f V_{\text{all}}(f, j), & \text{CV}_j &= \frac{\text{Std}_f[V_{\text{all}}(f, j)]}{\mu_j}. \end{aligned} \quad (11)$$

Selecting the K_s positions in $\mathcal{E}$ with the largest $\mu_j / (1 + \text{CV}_j)$ gives Equation 6. The core and anchor sets are disjoint, giving exactly $K_c + K_s$ retained tokens per frame. Only anchor positions are shared; their representations remain frame-specific.

A.2 ATTENTION PROFILING IN PILOT

Let $x_{\ell,h}(i, f, j)$ be the logit between action query i and future-frame key (f, j) , and let $\lambda_{\ell,h}(i)$ be the original log-sum-exp normalizer over all visible keys. Pilot directly computes

$$S_\ell(f, j) = \frac{1}{N_h} \sum_{i \in \mathcal{I}_f} \sum_{h=1}^{N_h} \alpha_{f,i} \exp(x_{\ell,h}(i, f, j) - \lambda_{\ell,h}(i)). \quad (12)$$

Reusing the original normalizer preserves the attention mass in R_ℓ ; normalization over future keys alone would change layer quality. The scorer follows the backbone’s query–key normalization, positional encoding, scaling, head mapping, and visibility constraints. It batches the relevant query–key products across frames and reduces over heads and aligned queries directly, storing only $S_\ell(f, j)$. No additional network forward, full attention matrix, or value-weighted output is needed.

B EXPERIMENTAL DETAILS

B.1 BENCHMARKS AND EVALUATION PROTOCOLS

LIBERO (Liu et al., 2023): four suites (Spatial, Object, Goal, Long), each with 10 tasks and 50 rollouts per task (2,000 total). **RoboLab-120** (Yang et al., 2026): 64 Simple, 39 Moderate, and 17 Complex tasks, each with 10 rollouts (1,200 total). **LIBERO-Plus** (Fei et al., 2025): the same 40 base tasks, with seven perturbation categories (background, camera, language, lighting, layout, robot initial state, and sensor noise). Two distinct variants per task and category, with one rollout each, yield 560 rollouts (80 per category; 140 per suite). All compared methods use identical selected variants, initial-state indices, and environment and model seeds.

B.2 IMPLEMENTATION DETAILS

Table 4 combines backbone and sparse-inference settings. Cosmos models use one wrist and two smaller third-person views; FastWAM-Joint uses one wrist and one third-person view. For Cosmos, instruction representations are cached across action chunks and visual conditioning is updated with each observation.

Table 4: Backbone and default Sparse-WAM configurations.

Configuration	Cosmos 3 Nano	Cosmos 3 Edge	FastWAM-Joint
Transformer layers	36	28	30
Camera views	3	3	2
Tokens per future frame	360	340	98
Future latent frames	8	8	2
Denoising steps	4	4	10
Guidance scale	3.0	3.0	1.0
Core tokens K_c	80	80	19
Shared anchors K_s	104	104	12
Retained tokens $K_c + K_s$	184	184	31

All backbones use an action horizon of 32, noise schedule shift 5.0, six informative layers, and CV penalty coefficient 1.0. The initial dense conditional forward ($\tau_d = 0$) collects statistics from all layers; core scores use the six highest-quality layers and anchor scores use all layers. Budgets are fixed, but layers and positions are selected online for each action chunk. Each frame has four aligned queries in Cosmos and sixteen in FastWAM-Joint. Central-half queries receive twice the weight of outer-quarter queries: normalized weights are $(1, 2, 2, 1)/6$ for Cosmos, and $1/12$ per central query and $1/24$ per outer query for FastWAM-Joint.

B.3 COMPARATIVE METHODS

ToCa caches only future visual tokens, retaining full observation and action computation. Cached steps reuse attention residuals and selectively update MLP features. Dense refresh steps are $\{0, 2\}$ for four-step Cosmos and $\{0, 4, 9\}$ for ten-step FastWAM-Joint.

WorldCache reuses or extrapolates both future visual and action predictions, with separate history per action chunk. Cosmos uses three initial dense steps followed by one cached step; FastWAM-Joint uses three initial dense steps, six cached steps, and a final dense step.

SpecPrune-VLA broadcasts observation-based spatial selections to all future frames and retains all observation and action tokens. The initial conditional forward builds a layer-wise pruning plan reused across denoising forwards; pruned tokens retain their exit-layer hidden states, restored before the output heads.

C³ache alternates dense refresh chunks with cache-reuse chunks, reusing joint visual-action Transformer residuals at matching denoising steps. Reuse covers the first two of four Cosmos steps or first five of ten FastWAM-Joint steps; embeddings, output heads, and sampler updates remain active.

B.4 TIMING DETAILS

We time Cosmos 3 Edge on an NVIDIA RTX 4090 with batch size 1, BF16, four denoising steps, and identical recorded inputs and seeds. Five warm-up runs precede 30 CUDA-event measurements, synchronized at both boundaries. Reported latencies are medians, except C³ache, whose latency is amortized over complete refresh/reuse cycles. Timing includes observation encoding, initial profiling, online scoring and selection, packing and restoration, cache operations, and sampler updates; it excludes model loading, compilation warm-up, CPU output transfers, video decoding, RPC, and simulation. The main tables use optimized accelerated methods against dense eager inference. Table 5 distinguishes sparse-inference gains from backend gains: Sparse-WAM achieves 1.55 $\times$ under eager execution, 1.85 $\times$ with optimizations relative to dense eager, and 1.56 $\times$ relative to equally optimized dense inference. Scoring and selection overhead is included throughout.

Table 5: Cosmos 3 Edge latency per action chunk. Panel (a) uses dense eager as the reference; panel (b) enables CUDA Graphs and `torch.compile` for every method, including dense inference. SpecPrune-VLA retains at least 184 future tokens per frame.

Configuration / Method	Latency (ms)	Speedup
<i>(a) Execution-backend ablation</i>		
Dense eager	859.31	1.00 $\times$
Sparse-WAM eager	555.55	1.55 $\times$
+ CUDA Graph	467.16	1.84 $\times$
+ <code>torch.compile</code>	464.95	1.85 $\times$
<i>(b) Matched-backend comparison</i>		
Dense (optimized)	727.44	1.00 $\times$
ToCa	687.03	1.06 $\times$
WorldCache	562.10	1.29 $\times$
SpecPrune-VLA	576.83	1.26 $\times$
C ³ ache	569.61	1.28 $\times$
Sparse-WAM	464.95	1.56 $\times$

B.5 REAL-WORLD FINETUNING AND EVALUATION

We fully fine-tune FastWAM-Joint on eight H800 GPUs, with batch size 8 per GPU (64 global), gradient accumulation 1, learning rate 10^{-4} , cosine scheduling, weight decay 0.01, BF16, and gradient clipping 1.0. Each prediction uses a single observation without history and produces 32 actions. Training budgets are 15,000 steps for packing three objects into a container, 20,000 for mouse-battery assembly, and 15,000 for stacking three cups. Dense and sparse variants are evaluated on these same tasks; Table 3 reports their success rates and latency.

C EXTENDED EXPERIMENTS

The following analyses quantify robustness, selection reuse, and the attention consistency motivating the method.

C.1 ROBUSTNESS ANALYSIS

Table 6 reports the seven perturbation categories of the LIBERO-Plus subset specified in Appendix B.1. Sparse-WAM reaches 63.04% success versus 70.89% for dense FastWAM-Joint and 51.96% for action-only FastWAM. Its latency falls from 431.7 to 207.2 ms ($2.08\times$), at a 7.85-percentage-point loss relative to dense inference. On this subset, sparse imagination preserves more robustness than removing imagination, but remains less robust than dense inference.

Table 6: LIBERO-Plus success rates (%) and generation latency per action chunk.

Method	Success Rate (%)								Latency (ms)
	Camera	Robot	Lang.	Light	Backg.	Noise	Layout	Avg.	
FastWAM-Joint	51.25	51.25	87.50	95.00	66.25	61.25	83.75	70.89	431.7
+ Sparse-WAM	31.25	46.25	85.00	92.50	57.50	47.50	81.25	63.04	207.2
FastWAM (action-only)	26.25	41.25	63.75	77.50	57.50	36.25	61.25	51.96	90.1

C.2 SELECTION REUSE AND REFRESH FREQUENCY

On Cosmos 3 Nano Policy, one full-refresh step in the four-step schedule yields 35.5% success and a $1.81\times$ speedup relative to refreshing at every denoising step (36.8% success). More frequent refreshes improve success but reduce acceleration. This comparison supports the default of scoring once per action chunk and reusing selections in later steps.

C.3 CROSS-STEP AND CROSS-FRAME ATTENTION OVERLAP

For normalized spatial attention distributions P and Q , we use

$$\text{Overlap}(P, Q) = \sum_i \min(P_i, Q_i). \quad (13)$$

Higher values indicate greater spatial agreement. Mean consecutive-step overlap is 81.11% on Cosmos 3 Edge and 97.97% on FastWAM-Joint; mean cross-frame overlap is lower, at 68.89% and 81.85%, respectively. These measurements support reusing selections across denoising steps while choosing core positions separately for each future frame.